

Deploy and Monitor Your First LangChain Service

FinQuery Analyzer: Docker, AWS ECR, Kubernetes, OpenTelemetry and Prometheus

Requirement	Deliverable	Where
1	Containerization and deployment assets	Dockerfile, k8s/deployment.yaml, k8s/service.yaml (Section 1)
2	Observability evidence	Section 2 (metrics export and chart from a live load test)
3	Observability reflection (150-250 words)	Section 3
4	Limitations and future improvements	Section 4

What was run and what was not

- **Run for real:** the instrumented service (FastAPI + OpenTelemetry + prometheus_client), a 300-request load test, and the /metrics export shown in Section 2. Manifests were checked for valid YAML.
- **Not run in the authoring environment:** docker build, the ECR push and kubectl apply (no Docker, AWS or cluster available there). Exact commands are in README.md.
- **Stand-in app:** the course starter finquery_analyzer.py was not provided, so the service uses a simulated LLM (200-800 ms latency, 5% injected 429-style failures). Swap in your real LangChain chain.

1. Containerization and Deployment Assets

How these assets show a production-minded deployment

- **Slim, reproducible image:** python:3.11-slim, pinned requirements installed before code is copied so the dependency layer is cached; .dockerignore keeps .git, .env and local files out.
- **Non-root, no secrets:** the container runs as UID 10001; API keys come from a Kubernetes Secret via secretKeyRef, never baked into the image or manifest.
- **Health and safety:** separate liveness (/healthz) and readiness (/readyz) probes, resource requests and limits, runAsNonRoot and no privilege escalation.
- **Scaling-ready:** stateless service, replicas: 1 to start, labels shared by Deployment and Service so an HPA can be attached later; a ClusterIP Service maps port 80 to the container port.
- **Monitoring built in:** Prometheus scrape annotations on the pod template plus a scrape config for annotation-based discovery.

Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

# Install pinned dependencies first so this layer is cached between code changes
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY finquery_analyzer.py .

# Run as a non-root user
RUN useradd --create-home --uid 10001 appuser && chown -R appuser /app
USER appuser

EXPOSE 8000

# No secrets baked in: OPENAI_API_KEY etc. are injected at runtime (K8s Secret)
CMD ["uvicorn", "finquery_analyzer:app", "--host", "0.0.0.0", "--port", "8000"]
```

.dockerignore

```
.git
.gitignore
__pycache__
*.pyc
.venv/
.env
evidence/
k8s/
*.md
```

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: finquery-analyzer
  labels:
    app: finquery-analyzer
spec:
  replicas: 1                                # start at 1; scale with an HPA once metrics look healthy
```

```

selector:
  matchLabels:
    app: finquery-analyzer
template:
  metadata:
    labels:
      app: finquery-analyzer
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/path: /metrics
      prometheus.io/port: "8000"
  spec:
    containers:
      - name: analyzer
        # Replace ACCOUNT_ID and REGION with your values
        image: ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/finquery-analyzer:1.0
        ports:
          - name: http
            containerPort: 8000
        env:
          - name: OPENAI_API_KEY          # from a Secret, never hard-coded
            valueFrom:
              secretKeyRef:
                name: finquery-secrets
                key: openai-api-key
                optional: true            # the stand-in LLM does not need it
          - name: OTEL_EXPORTER_OTLP_ENDPOINT # remove if no collector is deployed
            value: http://otel-collector.monitoring:4318
        readinessProbe:
          httpGet: { path: /readyz, port: http }
          initialDelaySeconds: 3
          periodSeconds: 5
        livenessProbe:
          httpGet: { path: /healthz, port: http }
          initialDelaySeconds: 10
          periodSeconds: 10
        resources:
          requests: { cpu: 250m, memory: 256Mi }
          limits: { cpu: "1", memory: 512Mi }
        securityContext:
          runAsNonRoot: true
          allowPrivilegeEscalation: false

```

k8s/service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: finquery-analyzer
  labels:
    app: finquery-analyzer
spec:
  type: ClusterIP          # use LoadBalancer if you need external access
  selector:
    app: finquery-analyzer # matches the Deployment pod labels
  ports:
    - name: http
      port: 80              # service port
      targetPort: http      # container port 8000

```

k8s/prometheus-scrape-config.yaml

```

# Add under scrape_configs in prometheus.yml (annotation-based discovery, matches deployment.yaml)
scrape_configs:
  - job_name: finquery-analyzer
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_label_app]
        action: keep
        regex: finquery-analyzer
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
        target_label: __metrics_path__
        regex: (.+)

```

```
- source_labels: [__address__, __meta_kubernetes_pod_annotation_prometheus_io_port]
  regex: ([^:]+)(?:\d+)?;(\d+)
  replacement: $1:$2
  target_label: __address__
```

finquery_analyzer.py (instrumented service)

"""FinQuery analyzer: LangChain-style regulatory-filing analysis service, instrumented with OpenTelemetry (traces) and prometheus_client (metrics).

Stand-in for the course starter: the LLM is simulated (latency + occasional 429) so the service runs without an API key. Replace `call\_llm` with a real LangChain chain in production.

```
Endpoints: POST /analyze   GET /healthz (liveness)   GET /readyz (readiness)   GET /metrics
"""
import asyncio, logging, os, random, time

from fastapi import FastAPI, HTTPException, Response
from pydantic import BaseModel, Field
from prometheus_client import CONTENT_TYPE_LATEST, Counter, Gauge, Histogram, generate_latest
from opentelemetry import trace
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor, ConsoleSpanExporter

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("finquery")

# ----- OpenTelemetry tracing (initialised first, at startup) -----
provider = TracerProvider(resource=Resource.create({"service.name": "finquery-analyzer"}))
if os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT"):
    # send to an OTEL Collector when configured
    from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
    provider.add_span_processor(BatchSpanProcessor(OTLPSpanExporter()))
elif os.getenv("OTEL_CONSOLE_TRACES") == "1":
    # local debugging
    provider.add_span_processor(BatchSpanProcessor(ConsoleSpanExporter()))
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("finquery.analyzer")

# ----- Prometheus metrics -----
REQUESTS = Counter("finquery_requests_total", "Analyze requests", ["status"])
LATENCY = Histogram("finquery_request_duration_seconds", "End-to-end request latency",
                    buckets=(0.05, 0.1, 0.25, 0.5, 0.75, 1, 1.5, 2.5, 5, 10))
STAGE = Histogram("finquery_stage_duration_seconds", "Per-stage latency", ["stage"],
                  buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5))
INFLIGHT = Gauge("finquery_inflight_requests", "Requests currently being processed")
LLM_ERRORS = Counter("finquery_llm_errors_total", "Failed LLM calls")

LLM_ERROR_RATE = float(os.getenv("LLM_ERROR_RATE", "0.05"))
app = FastAPI(title="FinQuery Analyzer")
ready = True

class AnalyzeRequest(BaseModel):
    document_id: str
    text: str = Field(min_length=1, max_length=200_000)

class AnalyzeResponse(BaseModel):
    document_id: str
    summary: str
    risk_flags: list[str]
    latency_ms: int

async def timed(stage: str, coro):
    """Run one pipeline stage inside an OTEL span and record its latency histogram."""
    with tracer.start_as_current_span(stage):
        t0 = time.perf_counter()
        try:
            return await coro
        finally:
            STAGE.labels(stage).observe(time.perf_counter() - t0)

async def fetch_document(doc_id: str, text: str) -> str:
    # CPU/IO-ish stage
    await asyncio.sleep(random.uniform(0.02, 0.06))
    return text.strip()

async def call_llm(text: str) -> str:
    # I/O-bound stage (simulated)
    await asyncio.sleep(random.uniform(0.2, 0.8))
    if random.random() < LLM_ERROR_RATE:
        LLM_ERRORS.inc()
        raise RuntimeError("simulated upstream 429")
    return f"Analysis: {text[:120]}"
```

```

async def summarize(raw: str) -> tuple[str, list[str]]:          # logic stage
    await asyncio.sleep(random.uniform(0.005, 0.015))
    flags = [w for w in ("penalty", "restatement", "litigation") if w in raw.lower()]
    return " ".join(raw.split()[:25]), flags

@app.post("/analyze", response_model=AnalyzeResponse)
async def analyze(req: AnalyzeRequest):
    t0 = time.perf_counter()
    INFLIGHT.inc()
    try:
        with tracer.start_as_current_span("analyze_request") as span:
            span.set_attribute("document.id", req.document_id)
            text = await timed("fetch_document", fetch_document(req.document_id, req.text))
            raw = await timed("llm_call", call_llm(text))
            summary, flags = await timed("summarize", summarize(raw))
            REQUESTS.labels("ok").inc()
            return AnalyzeResponse(document_id=req.document_id, summary=summary, risk_flags=flags,
                                   latency_ms=int((time.perf_counter() - t0) * 1000))
    except Exception:
        REQUESTS.labels("error").inc()
        log.exception("analyze failed doc=%s", req.document_id) # detail in logs only
        raise HTTPException(status_code=502, detail="Analysis temporarily unavailable. Please retry.")
    finally:
        LATENCY.observe(time.perf_counter() - t0)
        INFLIGHT.dec()

@app.get("/healthz")
async def healthz():          # liveness: process is up
    return {"status": "ok"}

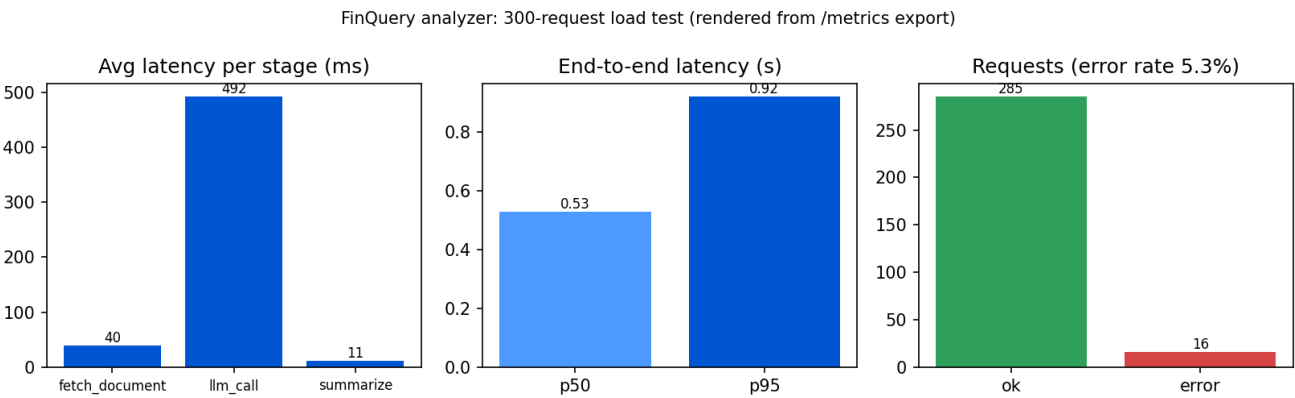
@app.get("/readyz")
async def readyz():           # readiness: safe to receive traffic
    if not ready:
        raise HTTPException(status_code=503, detail="not ready")
    return {"status": "ready"}

@app.get("/metrics")
async def metrics():
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)

```

2. Observability Evidence

Source: `/metrics` export from the running service after a 300-request load test (`python load_test.py 300`: 300 requests in 8.6s -> 34.7 req/s | ok=284 errors=16). The chart below was rendered from that export; it is not a Prometheus UI screenshot. After deploying to your cluster, replace or supplement it with a Prometheus or Grafana screenshot (queries in README.md).



Signal	Value	Source metric
Throughput	~34.7 req/s (300 requests in 8.6 s, concurrency 20)	finquery_requests_total
End-to-end latency	p50 about 0.53 s, p95 about 0.92 s (estimated from buckets)	finquery_request_duration_seconds
Stage latency (avg)	fetch_document 40 ms, llm_call 492 ms, summarize 11 ms	finquery_stage_duration_seconds
Errors	16 of 301 requests failed (5.3%), matching 16 LLM errors	finquery_requests_total{status=error}, finquery_llm_errors_total
Concurrency	in-flight gauge returned to 0 after the run	finquery_inflight_requests

Key signals illustrated

- Throughput, latency percentiles and error rate (the three signals the CTO asked for).
- The LLM stage dominates request time, which points to where to optimise.
- Failed requests return a clean 502 to the caller while details stay in the server log.

Raw `/metrics` excerpt

```
finquery_requests_total{status="ok"} 285.0
finquery_requests_total{status="error"} 16.0
finquery_request_duration_seconds_count 301.0
finquery_request_duration_seconds_sum 163.4718863119997
finquery_stage_duration_seconds_count{stage="fetch_document"} 301.0
finquery_stage_duration_seconds_sum{stage="fetch_document"} 12.174309897999947
finquery_stage_duration_seconds_count{stage="llm_call"} 301.0
finquery_stage_duration_seconds_sum{stage="llm_call"} 148.14118078700028
finquery_stage_duration_seconds_count{stage="summarize"} 285.0
finquery_stage_duration_seconds_sum{stage="summarize"} 3.0608552250001253
finquery_inflight_requests 0.0
finquery_llm_errors_total 16.0
```

3. Observability Reflection (220 words)

The per-stage latency histogram gave the most insight. In a 300-request local load test (concurrency 20, about 35 requests per second), end-to-end latency was roughly p50 0.5 s and p95 0.9 s, estimated from histogram buckets. The stage breakdown showed `llm_call` averaging about 490 ms, against about 40 ms for `fetch_document` and 11 ms for `summarize`, so the LLM call accounts for over 90% of request time and is the bottleneck. The error counter showed a 5.3% failure rate, and the LLM error counter matched it exactly, so the failures came from the upstream call and not from our own code. The traces add spans for each stage, so a slow individual filing can be tied to the stage responsible. These figures come from a local run with a simulated LLM, not from the cluster, so they demonstrate the method rather than production numbers. At 2,000+ documents a day (about 1.4 per minute on average, with bursts far higher), this setup lets operations alert on p95 latency, error rate and in-flight requests, notice upstream throttling before users do, and use latency and the in-flight gauge to decide when to add replicas with a Horizontal Pod Autoscaler. Because the bottleneck is waiting on the LLM, not CPU, retries with backoff and caching around `llm_call` would pay off before adding more hardware.

4. Limitations and Future Improvements

Limitations and risks

- **Unverified deployment path:** the image build, ECR push and cluster rollout still need to be run and checked (probes, image pull permissions, Service routing).
- **Simulated workload:** the LLM is simulated, so measured latencies and the 5% error rate are illustrative, not production figures; p50/p95 are bucket estimates.
- **Single replica and no autoscaling:** one pod is a single point of failure and cannot absorb a burst of 2,000+ documents.
- **Synchronous request path:** long LLM calls hold connections open; there are no retries, timeouts or caching around `llm_call` yet.
- **Monitoring gaps:** no alert rules, no dashboards in version control, traces only exported if a collector exists, no sampling policy, and Prometheus metrics are lost on pod restarts without durable storage.
- **Security:** no authentication, TLS or network policy on the service; filings may contain sensitive data that must not appear in logs or span attributes.

Recommended next steps

- Run the full build, push and deploy path, capture a real Prometheus or Grafana screenshot, and pin the image by digest.
- Add a Horizontal Pod Autoscaler (CPU plus a custom metric such as in-flight requests) and at least 2-3 replicas with a PodDisruptionBudget.
- Add timeouts, retries with exponential backoff and a cache around the LLM call; consider a queue (SQS or similar) so bursts are processed asynchronously.
- Define SLOs and alerts (p95 latency, error rate above threshold, saturation), provision Grafana dashboards as code, and deploy an OpenTelemetry Collector with tail sampling.
- Use IRSA or workload identity for ECR and secrets, add TLS and network policies, and scan images in CI.